

Module 12 – Containerization using Docker

Student Learning Notes

These notes are written based on my learning and understanding after studying the Docker module. The explanations are in simple language and summarize the important concepts I learned.

1. What I Learned in this Module

I learned the basics of Docker, why containerization is important, how Docker containers work, commonly used Docker commands, Docker Images, Docker Compose, Docker Engine, Docker Storage, Docker Networking and Container Orchestration using Kubernetes.

2. Introduction to Docker

Docker is a containerization platform used to package an application along with all its dependencies into a container. A Docker container runs the same way on different computers, making deployment easier and reducing environment-related issues.

3. Features of Docker

- Lightweight containers
- Fast application deployment
- Platform independence

- Easy scalability
- Resource efficient
- Simplified application distribution

4. Basic Docker Commands

`docker pull` - Downloads an image.

`docker images` - Lists downloaded images.

`docker run` - Creates and starts a container.

`docker ps` - Displays running containers.

`docker stop` - Stops a container.

`docker rm` - Removes a container.

`docker rmi` - Removes an image.

`docker exec` - Executes commands inside a running container.

5. Docker Run Command

I learned different ways of using `docker run` such as running a container with a custom name, running in detached mode, interactive mode, publishing ports and automatically removing the container after execution.

6. Docker Images

A Docker Image is a read-only template used to create containers. Images are built using layers, making storage efficient. Images can be created manually or by using a Dockerfile. Docker Hub is a common image repository.

7. Docker Compose

Docker Compose helps manage multiple containers using a single YAML configuration file. It simplifies starting, stopping and managing complete applications that use multiple services.

8. Docker Engine

Docker Engine is the core component of Docker. It consists of the Docker CLI, Docker Daemon and REST API. The CLI sends commands, the daemon performs operations and the API enables communication.

9. Docker Storage

Docker provides persistent storage using Volumes. Volumes allow data to remain available even after containers are removed. I also learned about storage drivers and volume management commands.

10. Docker Networking

Docker automatically creates default networks. Containers communicate through these networks. We can list, inspect and create custom networks depending on application requirements.

11. Container Orchestration

When many containers are used together, container orchestration helps manage them efficiently. Kubernetes is the most popular orchestration platform. It automates deployment, scaling, load balancing and monitoring of containers.

12. My Understanding

I understood that Docker solves the 'it works on my machine' problem by packaging applications with all required dependencies. Docker improves consistency, portability and deployment speed. Kubernetes becomes useful when applications contain many containers that need to be managed automatically.

13. Conclusion

This module gave me a clear understanding of Docker and its ecosystem. I learned how Docker simplifies software deployment, while Docker Compose and Kubernetes make it easier to manage modern cloud-native applications.